

# HCI 584 PROJECT SPEC: ELECTRIC VEHICLE CHARGING STATION LOCATOR.

## 1. GENERAL DESCRIPTION

The Electric Vehicle Charging Station Locator is a program designed to help EV drivers find nearby charging stations that meet their needs. The system will allow users to enter a location and search for charging stations based on factors such as distance, connector type, charging speed, price, and availability. The goal is to make finding a suitable charging station easier without requiring users to search through multiple charging-network apps or websites. This builds off of the original project sketch, which identified EV drivers, long-distance travelers, and new EV drivers as the primary users.

The application will use publicly available EV charging-station data from an API or dataset, such as the National Renewable Energy Laboratory (NREL) or Open Charge Map, both of which were identified as possible data sources in the original sketch. The app will process this data and filter the available stations based on the user's set preferences.

For version 1, the project will use a command-line interface (CLI) in Python. The user will enter their location and preferences through the terminal, and the program will return a list of matching charging stations. This will allow me to focus on getting the data flow and core functionality working before developing a graphical interface.

Ideally, the finished project would become a web or mobile-style application with an interactive map. A web interface using Flask or Streamlit could allow users to enter their location, select filters, view charging stations on a map, and select a station to see additional information. This would eventually provide the more visual experience described in the original sketch.

## 2. TASK VIGNETTES / USER ACTIVITY FLOW

### Vignette 1: Search for Charging Stations-

A driver with a low EV battery opens the charging-station locator and enters their current location. The system uses that location to search the charging-station dataset/API and returns nearby stations. The user can see basic information about each station, such as its name, distance, connector type, and charging speed.

#### Technical details:

- Users enter their location through the CLI.
- Location may be represented by a city, zip code, or latitude/longitude
- The program sends the location to the charging-station API
- API returns charging-station data
- Python processes the returned data
- Results are stored as dictionaries, lists, or a data frame
- Matching stations are displayed in the terminal

## Vignette 2: Filter Charging Stations-

The driver wants a faster charger that is compatible with their vehicle. They select filters for charging speed and connector type. The system removes stations that do not meet those requirements and displays the remaining options.

Technical details:

- Users select one or more filters.
- Possible filters include: distance, connector type, charging speed, price, availability
- Filtering occurs after the API data is retrieved.
- Python functions will process the station data.
- The results should update based on the selected filters.

## Vignette 3: View Station Details-

A driver selects one of the remaining stations. The system displays more detailed information so the driver can decide whether to use that station.

Technical details:

- User selects a station by number/name
- Program retrieves the corresponding station from the results
- Information displayed could include: station name, address, distance, connector type, charging speed, number of chargers, price, availability, operating hours.
- The result is displayed through the CLI in version 1

## Vignette 4: Adjust Search-

The driver decided that none of the results work for them. Instead of restarting the program, they change their search criteria, such as increasing the maximum distance or selecting a different connector type. The system runs the search again and displays the updated results.

Technical details:

- The application should support an interaction loop
- Users can return to the filter/search step
- Existing API data could potentially be reused instead of making a new request every time
- The loop continues until the user chooses to exit

## 3. Technical Flow

The version 1 flow should look something like this

User -> enter location -> get station data -> process station data -> apply user filters -> display results -> select a station -> display details -> search again? -> yes -> filters or no -> exit

### Potential Python Functions

get\_locations() -> get\_charging\_stations() -> process\_station\_data() -> filter\_stations() -> display\_stations() -> display\_station\_details() -> search\_again()

### Potential Data Types

#### Component:

- User Location
- API response
- Station Data
- Filters
- Search Results
- Station Details
- CLI output

#### Data:

- String / coordinates
- JSON
- List of dictionaries / DataFrame
- String, numbers, Boolean values
- List / DataFrame
- Dictionary
- Text

#### Possible externals

- Python - primary programming language
- EV charging API - provides station information
- Requests - retrieves API data
- Pandas - organizes and filters station data
- JSON - format for API responses
- Flask or Streamlit - possible future GUI
- Interactive Map Library - possible future map visualization

For version 1, I would like to keep the implementation limited to Python, API and CLI.

## 4. FINAL SELF-ASSESSMENT

1. What was the biggest or most unexpected change you had to make from your sketch?

- a. I think the biggest change was narrowing down to a CLI for version 1 instead of trying to build the complete mobile application immediately. I was honestly really scared and nervous about the idea of building an entire app, so narrowing it down lightened the stress a lot. The original sketch focused on an interactive map, but the project spec makes the underlying data flow and filtering function the priority.
2. How confident do you feel that you can implement the spec as it's written right now?
  - a. Like I mentioned, I'm super nervous. I've never done a project like this before but I see it as a great learning opportunity so I'm excited to give it a try.
3. What is the biggest potential problem that you NEED to solve (or you'll fail)?
  - a. I believe the biggest problem will be getting reliable data from an API and making sure the returned information contains the fields needed for the project.
4. What parts are you least familiar with and might need my help?
  - a. Working with the API, processing JSON data, calculating/filtering stations by distance, and eventually connecting the data to an interactive map or web interface.